



INSTALACIÓN · USO · OPERACIÓN

Observabilidad local, lista para operar.

Guía completa para levantar y utilizar SentinelMonitorIA en Windows con Docker Desktop, PostgreSQL, Redis, FastAPI y React/Vite.

Plataforma

Windows 10/11 + PowerShell

Modo recomendado

Docker Compose integrado

Estado

Local validado · AWS pendiente

ORIENTACIÓN

Cómo utilizar este manual

Este documento permite pasar de un repositorio recién clonado a una instancia local funcional, crear una identidad, emitir una API key, enviar telemetry y comprobar la salud del sistema.

Regla operativa principal. Ejecuta los comandos desde la raíz del repositorio. Elige un solo modo de frontend: manual con hot reload o Docker integrado en el puerto `3000`.

01	Resumen ejecutivo y alcance	02	Arquitectura local
03	Requisitos y preparación	04	Arranque rápido
05	Servicios y puertos	06	Primer uso y autenticación
07	API keys y telemetry	08	Configuración y entornos
09	Operación diaria	10	Redis Streams y worker
11	Pruebas y validación	12	Seguridad y troubleshooting
13	Agente Vector y E2E	14	AWS y límites actuales
15	Checklist de entrega		

Fuente de verdad técnica: README, backend README, runbook local e informe de validación incluidos en este repositorio.

VISIÓN GENERAL

Una plataforma local operable

SentinelMonitorIA es una plataforma de observabilidad y AIOps en evolución. Su recorrido funcional validado conecta identidad, organizaciones, API keys y procesamiento de telemetry en un stack local reproducible.

01

Identidad y organizaciones

Registro, login, refresh, logout, roles y membresías.

02

Ingesta de telemetry

Métricas, logs y eventos con validación y scopes.

03

Operación visible

Health checks, métricas, colas y dashboard.

Qué está validado

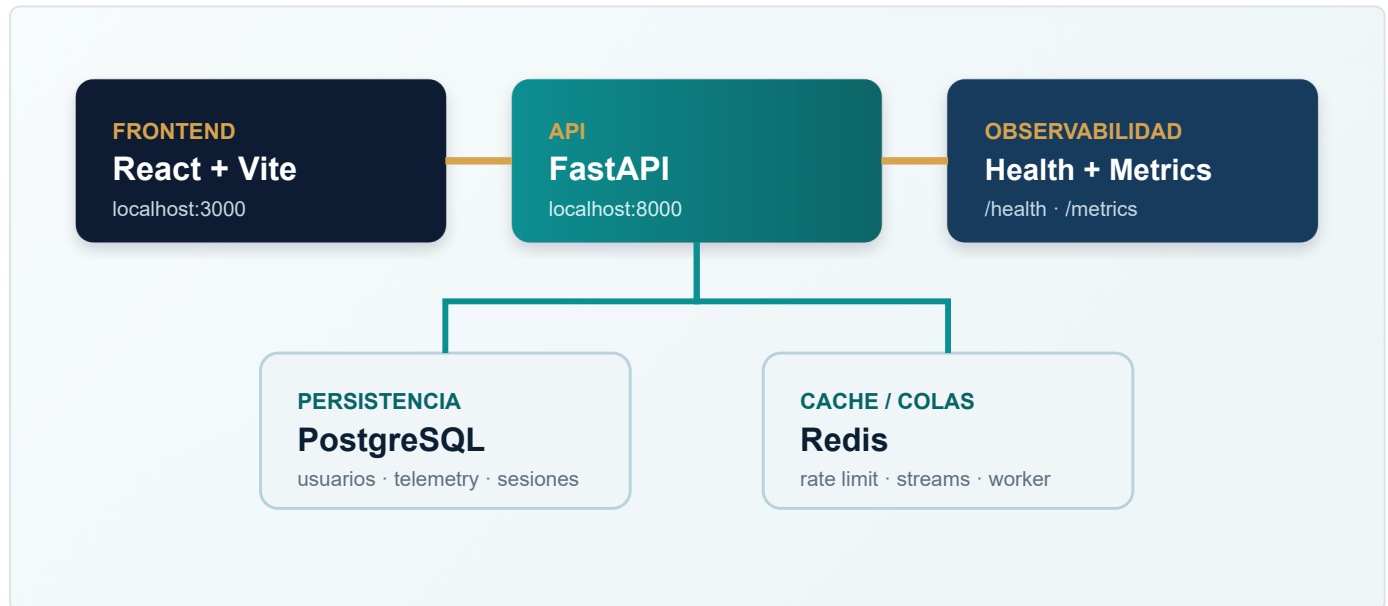
- Backend FastAPI con PostgreSQL y Redis.
- Frontend React/Vite manual y Docker.
- Cola mock predeterminada.
- Redis Streams y worker opcionales.
- API keys con digest SHA-256, scopes, rotación y revocación.
- Dockerfiles no-root y contextos sin `.env`.
- 19 pruebas backend correctas y 1 omitida por proveedor.
- 3 archivos frontend y 9 tests correctos.
- Build Vite 8.1.5 correcto.
- Auditoría npm sin vulnerabilidades.
- Smoke checks HTTP 200.
- Compose de desarrollo y local-production válido.

Fuera de alcance. AWS, SQS, S3, OpenSearch, ECS, DNS, certificados y costes cloud no se validan con este manual. CloudFormation sólo se entrega como diseño offline.

DISEÑO

Del usuario a la persistencia

El frontend habla con FastAPI mediante HTTP y CORS local. El backend valida sesiones y API keys, persiste en PostgreSQL y usa Redis para salud, rate limiting y colas opcionales.



Recorrido funcional

- 1 Identidad**
El usuario se registra, crea una organización y obtiene access/refresh tokens.
- 2 Conexión**
Desde **Connections** crea una API key asociada a la organización.
- 3 Ingesta**
Un agente envía un batch con Bearer API key a `/api/v1/telemetry`.
- 4 Procesamiento**
El proveedor mock procesa localmente; el override Redis añade streams, ACK, reintentos y DLQ.

PREPARACIÓN

Antes de abrir la aplicación

Software

- Docker Desktop con engine iniciado.
- Docker Compose v2.
- PowerShell.
- Node.js + npm para frontend manual.
- Git opcional.

Puertos

- 3000 frontend.
- 8000 API.
- 5432 PostgreSQL.
- 6379 Redis.
- 8080/8081 herramientas.

Clonar y comprobar Docker

```
git clone https://github.com/ramonesj/SentinelMonitorIA.git
Set-Location SentinelMonitorIA
./scripts/check-docker.ps1
```

El script comprueba CLI, Compose, daemon, WSL y ejecuta un contenedor de prueba. Si Docker no aparece en el PATH:

```
$env:Path = "$env:LOCALAPPDATA/Programs/DockerDesktop/resources/bin;$env:Path"
docker --version
docker compose version
docker info
```

No versionar secretos. Los archivos `backend/.env`, `env.local-production` y cualquier token deben permanecer locales. Los `.dockerignore` excluyen estos archivos de los builds.

OPERACIÓN INICIAL

Levantar el sistema en minutos

Opción A · Stack completo en Docker

1 Construir e iniciar

```
./scripts/start-local.ps1 -Build -Frontend
```

2 Esperar servicios saludables

```
docker compose -f backend/docker-compose.yml -f backend/docker-compose.frontend.yml ps
```

3 Ejecutar smoke checks

```
./scripts/test-local.ps1 -RequireFrontend
```

4 Abrir la interfaz

<http://localhost:3000>

Puerto 3000. Si existe un Vite manual activo, deténlo con `Ctrl+C` antes de iniciar ``sentinel-frontend``.

Opción B · Backend Docker + frontend manual

```
./scripts/start-local.ps1 -Build  
Push-Location frontend  
npm ci  
npm run dev  
Pop-Location
```

Usa esta opción si estás desarrollando componentes React y necesitas hot reload. El backend continúa en Docker y el navegador se conecta a ``http://localhost:8000``.

Detener sin borrar datos

```
docker compose -f backend/docker-compose.yml down
```

REFERENCIA RÁPIDA

Dónde encontrar cada componente

Componente	Dirección	Función	Modo
Frontend	localhost:3000	Login, dashboard y Connections	Manual o Docker
Backend	localhost:8000	Auth, health y telemetry	Docker
Swagger / OpenAPI	/api/v1/docs	Exploración de API	Development
PostgreSQL	localhost:5432	Usuarios, organizaciones y telemetry	Docker
Redis	localhost:6379	Cache, rate limit y colas	Docker
Adminer	localhost:8080	Gestión de PostgreSQL	Opcional local
Redis Commander	localhost:8081	Gestión de Redis	Opcional local
Metrics	/metrics	Formato Prometheus	Backend

Checks esenciales

```
Invoke-RestMethod http://localhost:8000/health
Invoke-RestMethod http://localhost:8000/api/v1/health/liveness
Invoke-RestMethod http://localhost:8000/api/v1/health/readiness
Invoke-RestMethod http://localhost:8000/api/v1/telemetry/health
Invoke-RestMethod http://localhost:8000/api/v1/telemetry/stats
Invoke-WebRequest http://localhost:3000/ -UseBasicParsing
```

Healthy

PostgreSQL, Redis y telemetry disponibles.

Ready

Dependencias preparadas para recibir tráfico.

HTTP 200

Frontend y endpoints públicos responden.

EXPERIENCIA DE USUARIO

Crear la primera conexión

- 1 Crear cuenta**
En el login, seleccionar **Create account** y completar nombre, username, email, contraseña y organización.
- 2 Iniciar sesión**
El backend entrega access y refresh token; el frontend restaura la sesión y renueva el access token cuando corresponde.
- 3 Revisar salud**
El dashboard consulta `/health`, `telemetry health`, `telemetry stats` y `/metrics`.
- 4 Abrir Connections**
Ver la organización activa y crear una API key con scope `telemetry:write`.
- 5 Guardar el secreto**
Copiar la key inmediatamente. El valor completo se muestra una sola vez.

Protección de credenciales. No pegues API keys o JWTs en tickets, capturas, logs, código fuente o archivos versionados. La lista de keys devuelve metadata, no secretos completos.

Roles y organizaciones

La organización inicial se crea durante el registro. Las reglas RBAC protegen la gestión de miembros, invitaciones, roles y API keys. Un usuario fuera de una organización recibe respuestas de autorización apropiadas y no puede leer sus datos.

Sesión

Logout, cambio de contraseña y refresh invalidan sesiones persistidas por `jti`.

API key

Digest SHA-256, expiración, scopes, rotación explícita y revocación.

INGESTA

Enviar el primer batch

Una API key necesita una organización y el scope `telemetry:write`. El batch debe incluir `metadata` y al menos una métrica, log o evento.

Ejemplo PowerShell

```
$base = "http://localhost:8000"
$apiKey = "API_KEY_COPIADA_UNA_SOLA_VEZ"
$headers = @{
    Authorization = "Bearer $apiKey"
    "Content-Type" = "application/json"
}
$payload = @{
    metadata = @{
        agent_id = "agent-local-001"
        hostname = "localhost"
        agent_version = "1.0.0"
        platform = "windows"
        architecture = "x64"
        tags = @{ environment = "local" }
    }
    metrics = @(
        @{ name = "system.cpu.usage"; value = 42.5; type = "gauge"; labels = @{} }
    )
    logs = @()
    events = @()
    batch_id = "local-batch-001"
} | ConvertTo-Json -Depth 8

Invoke-RestMethod -Uri "$base/api/v1/telemetry" -Method Post `
    -Headers $headers -Body $payload
```

202

Aceptado

El backend aceptó el batch.

telemetry:write

Permiso

Scope mínimo requerido para ingesta.

10.000

Límite

Máximo total de items por batch.

Consultar estadísticas

```
Invoke-RestMethod http://localhost:8000/api/v1/telemetry/health
Invoke-RestMethod http://localhost:8000/api/v1/telemetry/stats
```

Rotación y revocación

La rotación genera una replacement key y revoca la anterior. Tras revocar una key, una nueva ingesta con ese secreto debe devolver `401`. Un key sólo lectura debe devolver `403` al intentar escribir.

CONFIGURACIÓN

Elegir el perfil adecuado

Perfil	Cuándo usarlo	Características
Development	Trabajo diario y debugging	Hot reload backend, DEBUG, Swagger, herramientas Adminer/Redis Commander, cola mock.
Frontend Docker	Demo integrada o arranque de una sola orden	Imagen autocontenida, usuario `node`, rebuild explícito, puerto 3000.
Redis worker	Probar durabilidad y DLQ	Redis Streams, consumer group, ACK, reintentos y worker persistente.
Local-production	Ensayar restricciones cercanas a producción	DEBUG off, Redis autenticado, Alembic, no bind mount y capacidades reducidas.

Variables esenciales

Aplicación

`ENVIRONMENT`, `DEBUG`, `API_RELOAD`,
`API_CORS_ORIGINS`

Seguridad

`SECRET_KEY`, `JWT_SECRET_KEY`, expiraciones y Redis password en local-production.

Datos

`POSTGRES_HOST`, `POSTGRES_DB`, `POSTGRES_USER`,
`POSTGRES_PASSWORD`.

Cola

`QUEUE_PROVIDER=mock` por defecto; Redis Streams en override.

Importante. El campo válido es `ENVIRONMENT`; `APP_ENVIRONMENT` es histórico y no configura el backend.

Activar local-production

```
Copy-Item backend/.env.local-production.example backend/.env.local-production
# Editar y sustituir todos los placeholders

docker compose --env-file backend/.env.local-production `
  -f backend/docker-compose.local-production.yml config --quiet

docker compose --env-file backend/.env.local-production `
  -f backend/docker-compose.local-production.yml up -d --build
```

RUNBOOK

Rutina operativa segura

Estado y logs

```
docker compose -f backend/docker-compose.yml ps
docker compose -f backend/docker-compose.yml logs --tail 200 backend postgres redis
docker compose -f backend/docker-compose.yml logs -f backend
./scripts/test-local.ps1
```

Parada normal

```
docker compose -f backend/docker-compose.yml down
```

Esta orden conserva los volúmenes `postgres\_data`, `redis\_data` y `backend\_logs`. Para iniciar de nuevo:

```
./scripts/start-local.ps1
```

Migraciones

```
docker exec sentinel-backend alembic upgrade head
docker exec sentinel-backend alembic current
docker exec sentinel-postgres psql -U sentinel -d sentinelmonitoria -tAc "SELECT version_num FROM alembic_version;"
```

Limpieza destructiva

```
./scripts/start-local.ps1 -Clean
```

Acción destructiva. `-Clean` ejecuta `down -v` y elimina usuarios, organizaciones, tokens y telemetry. No lo uses para resolver un error sin confirmar primero que puedes perder el estado local.

Administración de datos

Adminer usa servidor `postgres` desde la red Docker y `localhost` desde Windows. Redis Commander se conecta al servicio `redis`. Estas herramientas son de desarrollo y no deben exponerse fuera de localhost.

COLAS

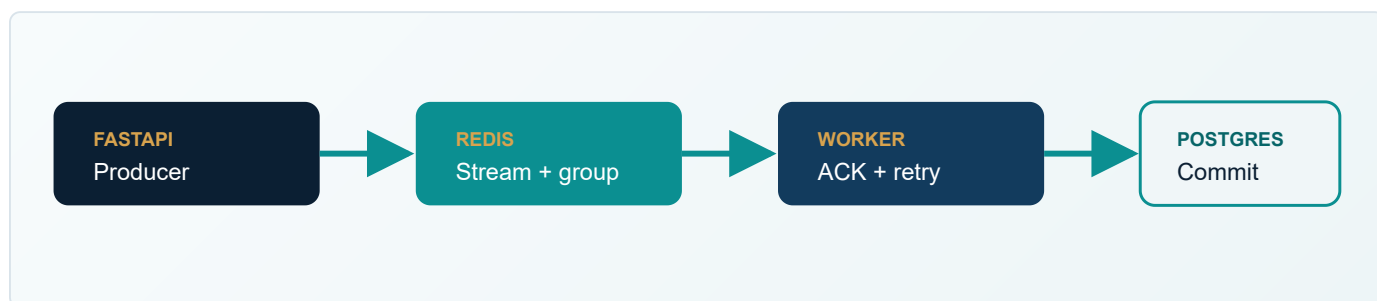
Pasar de mock a procesamiento persistente

El proveedor mock es la ruta predeterminada. Para ensayar procesamiento durable, activar el override Redis/worker sin eliminar los volúmenes existentes.

```
docker compose -f backend/docker-compose.yml `
  -f backend/docker-compose.redis-worker.yml config --quiet

docker compose -f backend/docker-compose.yml `
  -f backend/docker-compose.redis-worker.yml up -d --build backend worker

docker compose -f backend/docker-compose.yml `
  -f backend/docker-compose.redis-worker.yml ps
```



Comprobaciones

```
Invoke-WebRequest http://localhost:8000/api/v1/telemetry/health
docker exec sentinel-redis redis-cli XPENDING `
  sentinel:stream:telemetry sentinel-telemetry-workers
docker exec sentinel-postgres psql -U sentinel -d sentinelmonitoria `
  -c "SELECT status, count(*) FROM telemetrybatch GROUP BY status;"
```

- El worker usa consumer group y recupera pendientes con `XAUTOCLAIM`.
- Confirma con `XACK` después del commit PostgreSQL.
- Los fallos agotados se conservan en dead-letter.
- El replay DLQ es idempotente y conserva la entrada original.

No borrar Redis completo para resolver un `429` o revisar una cola. Diagnosticar primero las claves y el stream afectados.

EVIDENCIA

Demostrar que el entorno funciona

Frontend

```
Push-Location frontend
npm ci
npm test
npm run build
npm audit --audit-level=high
Pop-Location
```

Backend

```
docker exec sentinel-backend pytest -q
```

Compose, smoke y formato

```
./scripts/test-local.ps1 -RequireFrontend
docker compose -f backend/docker-compose.yml config --quiet
docker compose -f backend/docker-compose.yml `
  -f backend/docker-compose.frontend.yml config --quiet
docker compose --env-file backend/.env.local-production.example `
  -f backend/docker-compose.local-production.yml config --quiet
git diff --check
```

Prueba	Resultado validado
Frontend tests	3 archivos · 9 tests
Backend pytest	19 correctos · 1 omitido por Redis worker
npm audit	0 vulnerabilidades
Health/metrics/telemetry	HTTP 200
Build Docker frontend	Vite 8.1.5 · usuario node
Build Docker backend	usuario sentinel

La prueba omitida requiere levantar el proveedor Redis Streams y el worker persistente; no representa un fallo del stack mock predeterminado.

SEGURIDAD

Resolver problemas sin perder datos

Principios obligatorios

- ☐ No subir `.env`, API keys, JWTs o certificados.
- ☐ Cambiar secretos antes de un entorno compartido.
- ☐ Usar HTTPS fuera de localhost.
- ☐ No exponer Redis, Adminer, Redis Commander o Swagger fuera de desarrollo.
- ☐ Mantener CORS restringido.
- ☐ No imprimir tokens en logs.
- ☐ Usar `docker compose down`, no `down -v`, para paradas normales.

Diagnóstico rápido

Docker no responde

```
docker info
docker compose version
./scripts/check-docker.ps1
```

Backend no inicia

```
docker compose -f backend/docker-compose.yml logs --tail=200
backend
docker compose -f backend/docker-compose.yml ps
```

Frontend no responde

Liberar el puerto 3000. Elegir manual o Docker y reconstruir con `- Build`.

API desconectada

Comprobar `/health`, `VITE_API_BASE_URL` y CORS.

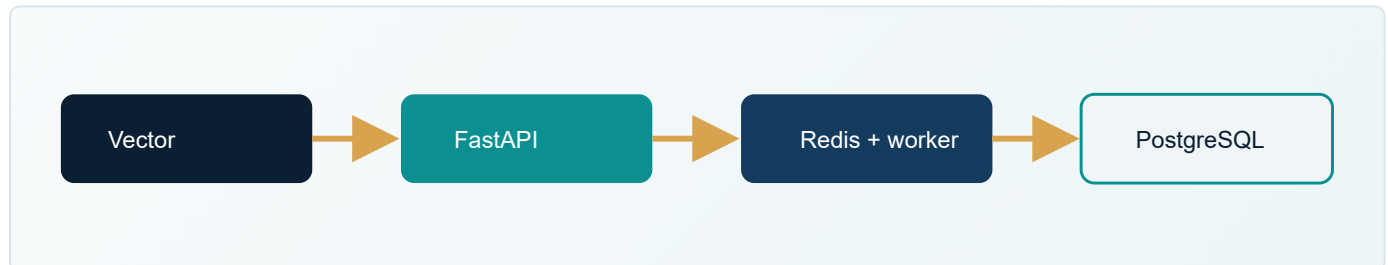
Errores frecuentes

Síntoma	Lectura correcta	Acción
401 en telemetry	Key ausente, expirada, revocada o sin organización.	Crear key con `telemetry:write` y usar Bearer.
403 al escribir	La key sólo tiene `telemetry:read` o no pertenece a la organización.	Revisar scopes y organización.
429 inesperado	Rate limiter o claves Redis previas.	Diagnosticar `rate_limit:*`; no borrar Redis completo.
Puerto ocupado	Otro perfil o Vite usa el puerto.	Detener el proceso correcto, no eliminar volúmenes.

INTEGRACIÓN

Validar el camino completo

La configuración normal de Vector está orientada a Linux porque utiliza host metrics, journald, archivos y logs Docker. Para Windows + Docker existe un flujo E2E aislado con fixture.



Flujo E2E aislado

```
./agent/scripts/generate-e2e-fixture.ps1
docker compose -f agent/deploy/docker-compose.e2e.yml up -d --build --force-recreate
docker exec sentinel-redis redis-cli XPENDING \
sentinel:stream:telemetry sentinel-telemetry-workers
```

El procedimiento completo para crear la API key, revisar los batches `processed` y repetir la prueba está en `agent/README.md`. No ejecutar el Compose auxiliar del agente junto al stack principal sin revisar puertos y redes.

Estado de soporte. El flujo E2E Windows + Docker está separado y validado; las fuentes host de Linux requieren un host Linux real con permisos y montajes adecuados.

GOBIERNO

Qué está listo y qué falta

Preparado offline

- Plan de foundation.
- VPC y subnets de ejemplo.
- Security groups.
- RDS y ElastiCache modelados.
- Repositorios ECR modelados.
- Parámetros de ejemplo.

Pendiente con cuenta

- Región, CIDR y costes.
- Secrets Manager e IAM.
- NAT y egress.
- ECS/Fargate y ALB.
- Route 53, ACM y WAF.
- Backups y rollback.

La carpeta `infra/cloudformation/` es diseño offline. No se han creado recursos ni se han validado cuotas, costes, conectividad o políticas en una cuenta AWS.

Validación futura

```
aws cloudformation validate-template \  
--template-body file://infra/cloudformation/sentinel-monitoria-foundation.yaml
```

No desplegar todavía. Antes de cualquier cuenta compartida hay que revisar secretos, costos, políticas IAM, exposición pública, backups, certificados y estrategia de rollback.

Fuentes del repositorio

- `README.md` : guía general.
- `backend/README.md` : API y worker.
- `docs/operations/local-runbook.md` : operación.
- `docs/operations/local-validation-report.md` : evidencia.
- `docs/deployment/cloudformation-plan.md` : plan cloud offline.

CIERRE

Checklist del operador

Antes de levantar

- ☐ Docker Desktop está ejecutándose.
- ☐ Los puertos requeridos están libres.
- ☐ No existe un secreto que vaya a versionarse.
- ☐ Se eligió frontend manual o Docker.

Después de levantar

- ☐ PostgreSQL, Redis y backend aparecen saludables.
- ☐ `/health`, readiness y telemetry health responden.
- ☐ Frontend responde HTTP 200.
- ☐ Usuario y organización fueron creados.
- ☐ API key tiene organización y ``telemetry:write``.
- ☐ Un batch válido devuelve ``202``.

Antes de compartir

- ☐ Secretos cambiados y fuera de Git.
- ☐ `npm audit` sin vulnerabilidades altas.
- ☐ Tests y build correctos.
- ☐ `git diff --check` limpio.
- ☐ AWS sigue marcado como pendiente hasta tener validación de cuenta.

Comando de oro:

```
./scripts/test-local.ps1 -RequireFrontend
```

Si todo responde correctamente, el entorno local está listo para demostración y desarrollo.